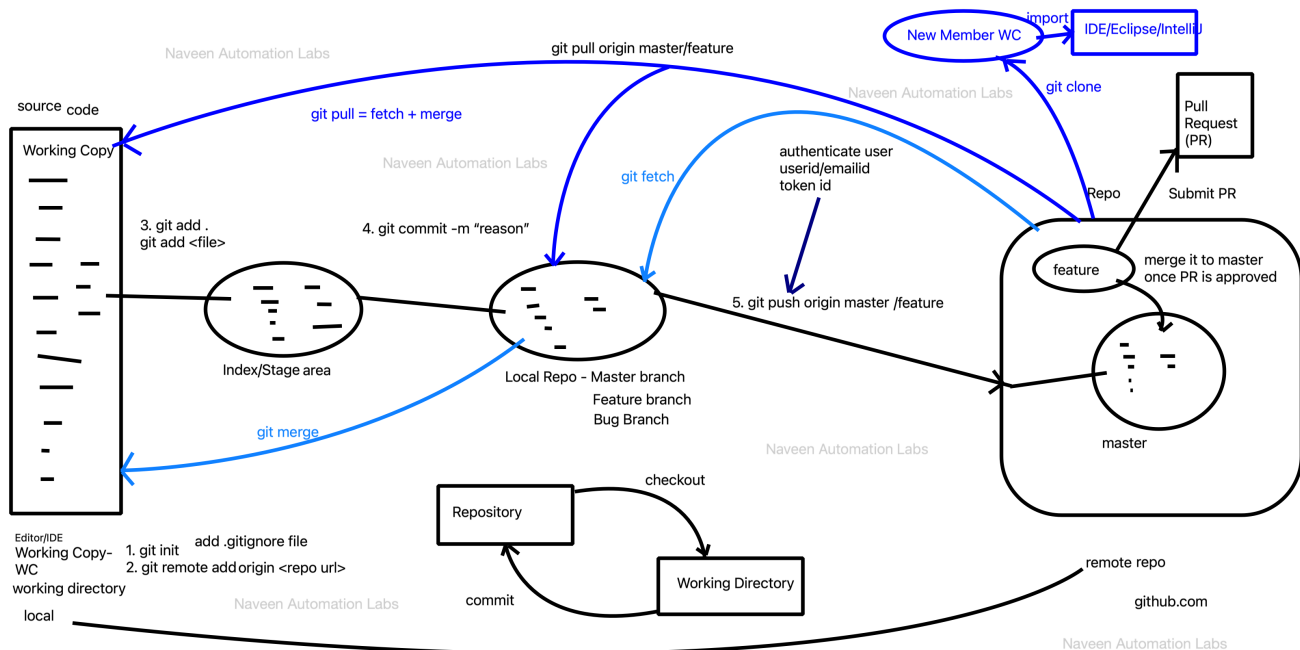


Git Workflow Process Documentation - Naveen Automation Labs



Introduction

This document explains the complete Git workflow process as shown in the workflow diagram, covering everything from initial repository setup to collaboration through pull requests and merging changes.

1. Initial Repository Setup

Local Repository Initialization

```
1. git init # Initialize a new Git repository
2. git remote add origin <repo url> # Connect to remote repository
```

Additional setup includes adding a **.gitignore** file to specify files that Git should ignore (like build artifacts, temporary files, and environment-specific files).

2. Working Copy Environment

The working copy represents your active development environment:

- Editor/IDE Working Copy (WC)
- Local working directory where you make changes to source code

3. The Basic Git Workflow

Stage and Commit Changes

```
3. git add .                # Stage all changes
   git add <file>           # Stage specific files
4. git commit -m "reason"    # Commit staged changes with a message
```

This process moves your changes through Git's three main areas:

1. **Working Directory** - Where you modify files
2. **Index/Stage Area** - Prepared changes ready to be committed
3. **Local Repository** - Committed changes in your local Git database

4. Branch Management

The diagram shows multiple branches:

- **Master branch** - The main production-ready code
- **Feature branch** - For developing new features
- **Bug branch** - For fixing issues

5. Remote Operations

Updating Your Local Repository

```
git fetch                  # Download changes from remote without
merging                    merging
git merge                   # Combine remote changes into your branch
git pull origin master/feature # Shorthand for fetch + merge
```

Sending Changes to Remote Repository

```
5. git push origin master /feature # Upload local commits to remote
repository
```

This requires:

- Authentication using user ID/email
- Token ID for secure access

6. Collaboration Workflow

Pull Request Based Development

1. Clone Repository

```
git clone <repository-url>
```

2. Create and Work on Feature Branch

```
git checkout -b feature
# Make changes
git add .
git commit -m "Feature implementation"
git push origin feature
```

3. Submit Pull Request (PR)

- Create PR through the repository platform (e.g., GitHub)
- Request review of your changes before they're merged to master

4. Review Process

- Team members review the code
- Feedback may require additional commits

5. Merge to Master

- Once PR is approved, changes are merged to the master branch
- This can be done through the platform interface or via Git commands

7. Additional Git Operations

Checking Out Repositories

```
git checkout # Switch branches or restore files
```

Repository to Working Directory Flow

The diagram shows the relationship between:

- Remote repository (e.g., GitHub.com)
- Local repository
- Working directory

Best Practices

1. **Commit Often** - Make small, focused commits that address a single issue or feature
2. **Write Clear Commit Messages** - Use descriptive commit messages explaining what and why
3. **Pull Before Push** - Always update your local repository before pushing changes
4. **Use Feature Branches** - Keep development work isolated from the main branch
5. **Review PRs Thoroughly** - Ensure code quality through careful review

Common Git Commands Reference

Command	Description
<code>git init</code>	Initialize a new repository
<code>git clone <url></code>	Clone a repository
<code>git add <file></code>	Stage changes
<code>git commit -m "message"</code>	Commit changes
<code>git pull</code>	Update local repository
<code>git push</code>	Send changes to remote
<code>git branch</code>	List branches
<code>git checkout <branch></code>	Switch branches
<code>git merge <branch></code>	Merge branches
<code>git status</code>	Show working tree status
<code>git log</code>	View commit history